Overdetermined Systems

Introduction to the experiment and guidance for reproducing the dissertation work

Purpose of this document. I have written this introduction to explain what I actually did in the overdetermined-system part of the dissertation before a reader has to deal with the Python code. The aim is not to teach numerical linear algebra in full. Instead, it gives enough background to understand what the programs are doing, what is being changed between experiments, what is measured, and why the larger runs were carried out on the University of Manchester CSF rather than on a normal laptop.

The repository contains two Python programs for this part of the work. Overdetermined.py contains the numerical benchmark. Overdetermined_plots.py is run afterwards to read the numerical CSV output and produce the figures and summary files. The CSV and PNG files are generated outputs; they do not need to be stored permanently with the source code because they can be recreated.

1. What the experiment is trying to answer

The dissertation compares four numerical approaches for solving large, sparse, overdetermined least-squares problems: Direct, Conjugate Gradient (CG), LSQR and LSMR. The main question is not simply which method gives an answer. I was interested in how their runtime, convergence behaviour and numerical accuracy change when the underlying problem is made larger, sparser, more poorly conditioned, differently structured, or differently regularised.

The problems are synthetic. This means that the matrix and the reference solution are created by the program rather than being read from a fixed real-world dataset. That is useful for a controlled experiment because one property can be changed while the others are held at the settings for that particular study.

2. The experimental structure

The table below is the correct way to read the experiment. There are six parameter studies, not six fixed matrices. Each study changes one main property and keeps the other settings fixed at the values defined for that study. The order shown for the matrix sizes is the order used in the experiment definition.

Study	Main quantity changed	Question being investigated
Condition Parameter	Condition parameter: 10, 100, 1000, 10000, 20000	How does increasing the intended conditioning difficulty affect convergence, runtime and accuracy?
Sparsity	Sparsity: 0.90, 0.95, 0.98, 0.99, 0.999	How does changing the amount of zero versus non-zero information affect the calculation?
Matrix Size	m: 20000, 30000, 25000; n fixed at 5000	How does increasing the number of equations affect computational cost and solver behaviour?
Lambda	lambda: 0, 1e-8, 1e-6, 1e-4, 1e-2, 1e-1, 1	How does the strength of regularisation change the solution and solver behaviour?
Sparsity Pattern	random, banded, block	Does the arrangement of non-zero entries matter, even when the broad sparsity setting is comparable?
Spectrum	linear, exponential, polynomial, clustered	Does the distribution of matrix scaling affect numerical behaviour?

Important: these six studies overlap in their underlying parameter space. They are different views of the same benchmark framework, rather than six unrelated sets of matrices. For reproducibility, the exact

configuration should be taken from the experiment definition in Overdetermined.py, not invented from the name of a folder or from a function default.

3. What is an overdetermined linear system?

A linear system can be written as A x = b. The matrix A contains the coefficients, x is the vector being estimated, and b is the right-hand side. If A has m rows and n columns, the system is overdetermined when m is greater than n. In simple terms, there are more equations or observations than unknown quantities.

The right-hand side in this experiment is deliberately noisy. The program first creates a known reference vector, forms A x_true, and then adds noise. The solver therefore cannot be expected to make every equation exactly zero. Instead, the quality of the solution is assessed using the residual and, because the reference vector is known, the relative error in the computed solution.

4. Parameters used in the dissertation experiments

The following values describe the parameter ranges used in the overdetermined experiment. This replaces the incorrect interpretation of the benchmark as a six-row list of fixed matrices. The matrix dimensions and other settings are selected by study.

Parameter	Values used	How it is used
Matrix size	n = 5000; m = 20000, 30000, 25000 in Matrix Size study	The Matrix Size study changes m while n remains fixed.
Condition Parameter	10, 100, 1000, 10000, 20000	Used to control the range of spectral scaling during matrix construction.
Sparsity	0.90, 0.95, 0.98, 0.99, 0.999	Controls the amount of zero versus non-zero information in the generated matrix.
Lambda	0, 1e-8, 1e-6, 1e-4, 1e-2, 1e-1, 1	Regularisation parameter used by the solver formulations.
Sparsity Pattern	random, banded, block	Changes where the non-zero entries are placed.
Spectrum	linear, exponential, polynomial, clustered	Changes how the scaling factors are distributed.

There is one deliberate exception to the common overdetermined dimensions. The Condition Parameter study uses m = 51000 and n = 5000, with sparsity 0.95, the random pattern, the exponential spectrum and lambda = 1. The other large studies use m = 20000 and n = 5000 unless the Matrix Size study is changing m.

5. Condition parameter versus actual condition number

I use the term Condition Parameter deliberately. The value supplied to the matrix generator controls the range of scaling factors, but it is not automatically the exact spectral condition number of the final sparse matrix. The realised condition number also depends on the matrix dimensions and sparsity structure.

This distinction was checked using a small diagnostic with 200 by 50 matrices, 95% sparsity and an exponential spectrum. For the random pattern, the prescribed condition parameters 10, 100, 1000 and 10000 produced measured spectral condition numbers of approximately 21.09, 177.44, 1702.66 and 16768.63 respectively. Additional tests with the banded and block patterns at condition parameter 1000 produced approximately 1278.63 and 1467.71. The point of the diagnostic is that the input parameter and the realised matrix condition number are related, but they are not the same quantity.

6. How the test matrix and right-hand side are created

The matrices are generated directly by the benchmark. No large matrix files are required as input. For each configuration, the program creates a sparse matrix with the requested dimensions and sparsity pattern and then applies the selected spectral scaling.

6.1 Sparse matrix construction

The sparsity setting controls how much of the matrix is zero. The implementation uses the sparsity value to derive a matrix density, with a lower density meaning fewer stored non-zero entries. The three supported patterns are random, banded and block. This matters because two matrices can contain a similar amount of non-zero information while having different computational behaviour because those entries are arranged differently.

6.2 Spectral scaling

The program creates scaling factors according to one of four choices: linear, exponential, polynomial or clustered. For the linear and exponential constructions, the scaling range runs from 1 towards 1 divided by the supplied condition parameter. The other spectral choices alter the distribution of the scales. The resulting scales are applied to the generated sparse matrix rather than directly prescribing the singular values of the final matrix.

6.3 Reference solution and noise

For each generated problem, the program creates x_true using a standard normal random-number generator. It then forms the clean right-hand side as A x_true. Noise is added afterwards. The dissertation uses a relative noise level of 0.01, with controlled random-number generation beginning from seed 42. Repeated runs use the corresponding seed progression, so the same run number can be compared across solver methods.

This is important when reproducing the results. If the matrix is regenerated with a different seed, or if the noise is produced differently, the solver is no longer being tested on the same problem instance. Small numerical differences can still occur because of the processor and numerical-library environment, but the experimental construction itself should not be changed.

7. What happens in one benchmark run?

- The program selects the dimensions and all other parameters for the current configuration.
- A sparse matrix A is generated and spectrally scaled.
- A known reference vector x_true is generated.
- The program forms A x_true and adds controlled noise to produce b.
- One of the four solver implementations is applied to the resulting regularised least-squares problem.
- The solver runtime, iteration count where applicable, residual and relative solution error are calculated.
- The complete parameter configuration is stored with the measured result.
- The process is repeated for the requested number of repetitions.

The standard benchmark uses 10 repeats. The repeated runs are not simply ten copies of the same random matrix: the controlled seed is advanced for each repeat. This gives a set of measurements from related but separately generated problem instances and allows mean behaviour to be reported.

8. The four numerical methods

The four methods are not interchangeable implementations of the same algorithm. They treat the regularised least-squares problem differently, which is central to the comparison.

Direct

The Direct implementation explicitly forms A^T A and the right-hand side A^T b. It then solves the regularised system A^T A + lambda I using a sparse direct solver. This approach is straightforward, but the construction and factorisation of the normal-equation matrix can require substantial memory for the largest problems.

Conjugate Gradient (CG)

In the supplied implementation, CG also explicitly forms A^T A. It constructs the regularised matrix H = A^T A + lambda I and then applies the conjugate-gradient algorithm to H. Therefore, the CG implementation in the repository should not be described as matrix-free. The matrix-free distinction belongs to alternative CG implementations, not to the code supplied here.

LSQR

LSQR works directly with A and b and does not explicitly construct A^T A. Regularisation is introduced through the damping parameter, implemented as the square root of lambda. The stopping tolerances are 1e-8 and the iteration limit is 5000.

LSMR

LSMR also works directly with the original sparse matrix and uses damping derived from lambda. As with LSQR, the supplied implementation uses 1e-8 stopping tolerances and a maximum of 5000 iterations.

9. Solver settings that belong to the experiment

Solver	How the problem is handled	Stopping / iteration setting
Direct	Explicit A^T A; sparse direct solve of A^T A + lambda I	Recorded as one solve
CG	Explicit A^T A; CG applied to A^T A + lambda I	rtol = 1e-8; atol = 0; maxiter = 5000
LSQR	Works directly with A and b; damping = sqrt(lambda)	atol = 1e-8; btol = 1e-8; iter_lim = 5000
LSMR	Works directly with A and b; damping = sqrt(lambda)	atol = 1e-8; btol = 1e-8; maxiter = 5000

> Reproduction point: do not change the solver implementation simply because another implementation might be more memory efficient. The comparison in the dissertation is based on the supplied implementations. In particular, the explicit construction of A^T A in Direct and CG is part of the computational setup that produced the reported results.

10. What is measured

Measurement	Meaning
Time (s)	Elapsed time measured for the solver call.
Iterations	Number of iterations for iterative solvers; Direct is recorded as one solve.
Residual	Norm-based measure of the mismatch between A x and b.
Relative Error	Distance between the computed x and the known x_true, divided by the norm of x_true.

11. How the six parameter studies should be read

Each folder is an analytical view of the same underlying benchmark. When a study changes one parameter, the other parameters are fixed at the settings specified for that study. The complete configuration is still written to the result rows, so a generated CSV can be traced back to the problem that produced it.

Condition Parameter study

This study varies the condition parameter through 10, 100, 1000, 10000 and 20000. The matrix dimensions are m = 51000 and n = 5000, with sparsity 0.95, the random pattern, the exponential spectrum and lambda = 1. The purpose is to see how increasing the intended spectral difficulty affects solver runtime, convergence and accuracy.

Sparsity study

This study keeps m = 20000 and n = 5000 and varies sparsity through 0.90, 0.95, 0.98, 0.99 and 0.999. The condition parameter is 1000, the pattern is random, the spectrum is exponential and lambda is 1. This isolates the effect of changing the amount of non-zero information.

Matrix Size study

This study keeps n = 5000 and varies m in the order 20000, 30000, 25000. The other main settings are condition parameter 1000, sparsity 0.95, random pattern, exponential spectrum and lambda = 1. The ordering is retained because it is the order used in the benchmark definition.

Lambda study

This study keeps m = 20000 and n = 5000, with condition parameter 1000, sparsity 0.95, random pattern and exponential spectrum. Lambda is varied through 0, 1e-8, 1e-6, 1e-4, 1e-2, 1e-1 and 1. This examines how regularisation changes the numerical problem and the solver behaviour.

Sparsity Pattern study

This study compares the random, banded and block constructions while keeping the other main settings fixed. It is intended to separate the effect of where the non-zero entries are located from the effect of simply changing how many non-zero entries there are.

Spectrum study

This study compares the linear, exponential, polynomial and clustered spectral constructions while keeping the main problem dimensions and other controls fixed. It asks whether the distribution of scaling factors matters in addition to the broad condition parameter.

> Do not turn these into six 'cases'. That interpretation loses the purpose of the experiment. The folders are study categories used to organise the analysis; the numerical program is the part that defines the actual parameter combinations and repeats.

12. From numerical results to plots

I separated the expensive numerical calculation from the plotting stage so that the figures can be recreated without running the large benchmark again. This is also why the repository does not need to keep every generated PNG and CSV file.

Stage 1 - Run Overdetermined.py

The numerical program generates the matrices, runs the selected solver experiments, and writes CSV result files. The result rows contain the parameter values as well as the measured quantities. The program also writes intermediate result files during the sweep so that a long run is less likely to leave no numerical record if a later configuration fails.

Stage 2 - Run Overdetermined_plots.py

The plotting program reads the benchmark CSV files. It groups the results by the relevant study parameter and solver, calculates summary statistics where required, and produces the plots used for the analysis. Because this stage reads existing numerical output, it can be rerun without regenerating the large matrices.

13. What the CSV files contain

Field	Purpose
m, n	Dimensions of the matrix.
Condition Parameter	Conditioning/scaling control used by the matrix generator.
Sparsity	Sparsity setting used to generate the matrix.
Sparsity Pattern	Random, banded or block construction.
Spectrum	Linear, exponential, polynomial or clustered construction.
Lambda	Regularisation parameter.
Solver	Direct, CG, LSQR or LSMR.
Run	Repeat number.
Time (s)	Measured solver runtime.
Iterations	Iteration count where applicable.
Residual	Residual norm measure.
Relative Error	Relative error with respect to x_true.

The generated files are therefore outputs rather than required inputs. For a clean repository, the source material can contain the Python programs and documentation while the CSV and PNG outputs are recreated when the experiment is run. A researcher who wants to compare reproduced values with the dissertation can retain the generated outputs separately.

14. Why the large experiment uses CSF and Slurm

The largest systems are not intended to be treated as ordinary desktop calculations. The sparse representation reduces storage compared with a dense matrix, but forming and solving the normal-equation system can still require substantial memory. The large benchmark was therefore run on the University of Manchester CSF using Slurm.

15. Reproducing the computational environment

The repository assumes that the person reproducing the work has legitimate access to the University of Manchester CSF. University login and account setup are not reproduced here; the current University instructions should be followed because those procedures can change.

The benchmark was developed using the Anaconda scientific Python environment used for the dissertation, with Python, NumPy, SciPy, pandas and Matplotlib. When reproducing timings, software versions should be recorded because Python versions, numerical libraries, BLAS/LAPACK implementations and processor architecture can affect performance.

16. Memory and Slurm resources

The large benchmark was organised around a 24-hour Slurm allocation. This should be understood as part of the original workflow rather than as a guarantee that every current CSF account can request exactly the same resource configuration.

For the large Direct-solver cases, an allocation below 128 GB of RAM should be treated as insufficient for reliable reproduction. This is a practical requirement observed for this benchmark, not a mathematical statement that every possible implementation of a sparse direct solver needs 128 GB.

The number of CPUs is user- and allocation-dependent. The person reproducing the work must check the current University/CSF limits and determine the CPU count, memory limit, wall time and appropriate Slurm partition available to their account. These values should not be copied blindly from an old job submission.

> In practice: before submitting a reproduction job, check the current CSF documentation for the memory that can be requested, the number of CPUs permitted, the available wall-time limits and the appropriate queue or partition. The important experimental constraint is that the resources must be sufficient for the selected cases, especially the Direct cases.

17. The Slurm organisation

The dissertation workflow used a 24-element Slurm array to organise the benchmark work. This is a description of how the runs were grouped; no historical Slurm job number is needed to reproduce the work. A current reproduction should create a new submission under the user's own valid CSF allocation.

The exact Slurm command, account syntax, partition name and CPU/memory directives should be checked against the current CSF environment. Those are infrastructure details and can change without changing the mathematical experiment.

18. Reproducibility rules

The following points are the ones I would keep unchanged when reproducing the dissertation experiment. They are intended to prevent a technically successful run from becoming a different experiment.

- Keep the study definitions unchanged. Use the values in Overdetermined.py rather than substituting smaller or easier values simply to make a local run finish.
- Keep the matrix dimensions unchanged. In particular, the Condition Parameter study uses m = 51000 and n = 5000, while the common large configuration uses m = 20000 and n = 5000.
- Keep the parameter order where relevant. The Matrix Size study uses m values 20000, 30000 and 25000 in that order.
- Keep the noise level and seed handling. The benchmark uses relative noise level 0.01 and starts from seed 42, with the repeat number advancing the seed.
- Keep the solver implementations unchanged. Direct and CG explicitly construct A^T A in the supplied implementation; LSQR and LSMR operate directly through A.
- Keep the solver tolerances and iteration limits unchanged. These settings affect both timing and the numerical results.
- Keep the plotting stage separate. Run the plotting program from the numerical CSV output rather than rebuilding matrices solely to obtain figures.
- Record the computational environment. CPU allocation, memory allocation, wall time, Python version and numerical-library versions should be recorded when reporting a reproduction.
- Check current CSF rules. University resource limits and available software environments can change after the dissertation was completed.

19. What a successful reproduction should produce

A successful numerical run should leave the benchmark CSV files containing the full parameter configuration, solver name, repeat number and measured performance values. The plotting stage should then be able to read those files and recreate the parameter-study figures and summary CSV files.

The reproduced timings do not have to be numerically identical to the dissertation timings. Runtime is affected by the CPU architecture, current system load, library versions and other environmental factors. What should remain consistent is the problem definition, solver implementation, numerical settings and experimental structure.

Likewise, a failed Direct run should not be 'fixed' by silently reducing the matrix size or changing the solver. If the requested case exceeds the available resources, the correct response is to check the current CSF allocation and resource limits and then rerun the same case with an appropriate allocation.

20. A simple mental model for the whole workflow

The easiest way to think about this section is as a controlled stress test. I create a family of sparse overdetermined problems, change one main characteristic at a time, and ask four different numerical methods to solve them. The program records how long they take, how many iterations they use where applicable, how closely their solution satisfies the equations, and how close the computed solution is to the known reference vector.

The six folders then provide six different ways of looking at those measurements: Condition Parameter, Sparsity, Matrix Size, Lambda, Sparsity Pattern and Spectrum. The numerical program is the source of

the data; the plotting program is the source of the figures.

21. What a new researcher needs before starting

A reader who has never run this type of experiment does not need to understand every line of Python before beginning. The important prerequisites are practical:

- A working Python/Anaconda environment with the packages required by the supplied programs.
- A copy of Overdetermined.py and Overdetermined_plots.py with their filenames and contents unchanged.
- Valid CSF access and knowledge of the University's current CSF login procedure.
- A current understanding of the memory, CPU, wall-time and partition limits available to the account.
- At least 128 GB of memory for the large Direct-solver cases, subject to the current University allocation rules.
- Enough wall time for the selected experiment; the dissertation workflow used a 24-hour allocation.
- The ability to submit and monitor a Slurm job, including a Slurm array where the full benchmark is being reproduced.
- The generated numerical CSV files before the plotting stage is attempted.

22. What should and should not be stored in the repository

The source repository is intended to make the computational work reproducible without becoming a permanent archive of every generated result. The useful permanent material is the Python implementation and the documentation explaining how it is used. Generated CSV and PNG files can be recreated from the code and can therefore be kept outside the repository when a clean technical-materials submission is preferred.

If generated results are retained for checking, they should be treated as outputs of a particular run and labelled with the computational environment and parameter configuration used to produce them. They should not be mistaken for additional input data required by the programs.

23. Final note on reproduction

The main thing to preserve is the experiment itself. The purpose of this documentation is not to force a future researcher to reproduce an old computer environment byte for byte. It is to make clear which choices define the dissertation experiment and which choices belong only to the computing infrastructure available at the time.

For the mathematical and numerical part, the supplied Overdetermined.py is the authoritative implementation. For the figures and summaries, Overdetermined_plots.py is the corresponding post-processing program. For CSF resources, the current University guidance is authoritative. Keeping those three points separate makes the repository much easier for another researcher to understand and reproduce.

24. Source basis for this introduction

This document has been revised against the experiment definitions and solver implementation contained in the dissertation material and supplied Python source. In particular, the parameter studies, matrix dimensions, condition-parameter values, sparsity values, lambda values, pattern choices, spectrum choices, noise level, seed handling and solver settings are taken from those materials. The resource guidance distinguishes the dissertation workflow from current CSF allocation rules so that infrastructure-specific details are not presented as permanent University guarantees.

Repository files referred to in this document:
Overdetermined.py
Overdetermined_plots.py
Overdetermined_Systems_Introduction.pdf
Overdetermined_Systems_Run_Instructions.pdf
Overdetermined_Systems_Requirements_and_Restrictions.pdf